FLAG

FLAG · MÓDULO 01 · SESSÃO 6

Front-End e Back-End

Quem faz o quê, HTML, CSS e JavaScript por dentro, APIs, JSON e bases de dados. E o computador pronto para HTML e CSS

2.ª feira, 12 out 2026 · 9:30–12:30 · MS Teams

João Gonçalves · joaogoncalves.net

FLAG

Boas práticas Live Training



Escolher um sítio calmo, sem distrações



Ser pontual



Desligar microfone. Ligar quando for para intervir



Manter a câmara ligada



Levantar a mão quando pretender participar



No intervalo, desligar a câmara e o microfone

RECAPITULAR A SESSÃO 5

Três perguntas para aquecer

1

O que faz o DNS?

Traduz nomes (flag.pt) em endereços IP.

2

404 ou 500: de quem é a culpa?

404 do cliente (não existe); 500 do servidor.

3

O que acrescenta o S de HTTPS?

Cifra: confidencialidade, integridade, autenticidade.

Hoje: o que o servidor envia, o que o browser faz com isso, e quem trabalha de cada lado.

DA SESSÃO 5 · PARTE 4

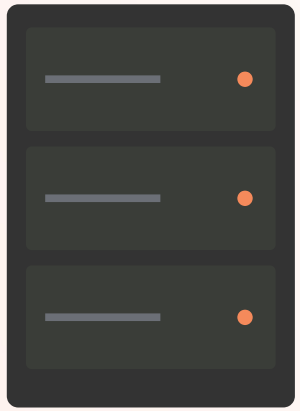
Onde vivem os sites

servidores · cloud · CDN · alojamento · do código ao site publicado

TEORIA → EXERCÍCIO 4.1 → EXERCÍCIO 5.1 (flag.pt + Enter)

FLAG

Um servidor é um computador e um programa



num datacenter

Ficheiros e dados

HTML, CSS, JS, imagens · bases de dados

Software servidor

Nginx, Apache ou Node.js: escuta numa porta e responde a pedidos HTTP

Sistema operativo

quase sempre Linux

Hardware

CPU, memória, disco, rede. Sem ecrã nem teclado, ligado 24 h

O vosso portátil também

Com o Live Server (módulo HTML) ou o Vite (módulo React), o VS Code arranca um pequeno servidor em localhost. É só para vocês: ninguém lá fora o vê.

Publicar é pôr os ficheiros num servidor que está sempre ligado e tem um IP público.

Quem trata de quê? (pizza as a service)

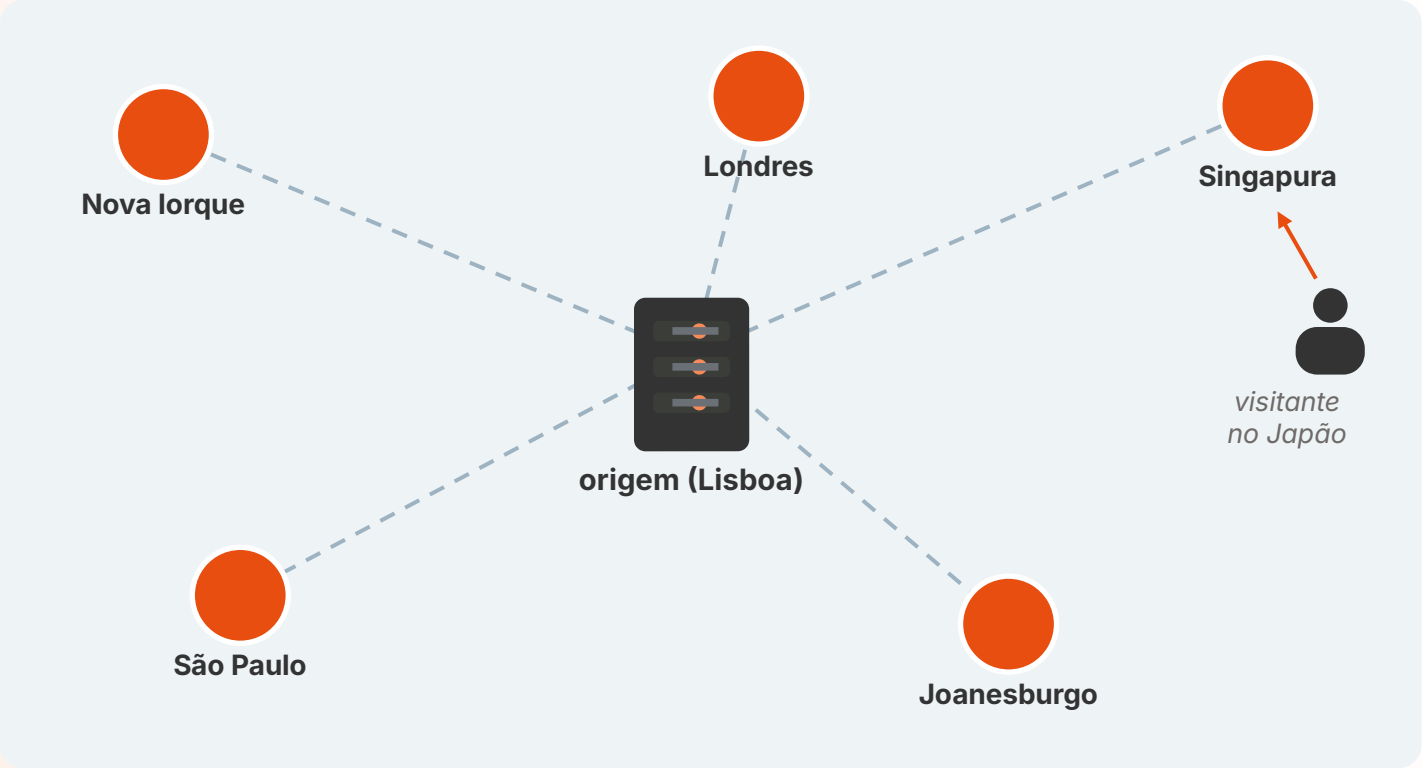
	Servidor próprio *pizza feita em casa*	IaaS · VPS *pizza congelada, forno vosso*	PaaS *entrega ao domicílio*	SaaS *jantar na pizzaria*
A vossa aplicação	**vocês**	**vocês**	**vocês**	fornecedor
Os dados	**vocês**	**vocês**	**vocês**	fornecedor
Runtime (Node, PHP)	**vocês**	**vocês**	fornecedor	fornecedor
Sistema operativo	**vocês**	**vocês**	fornecedor	fornecedor
Máquina	**vocês**	fornecedor	fornecedor	fornecedor
Rede e datacenter	**vocês**	fornecedor	fornecedor	fornecedor
Exemplos	**servidor na empresa**	**AWS EC2, Hetzner, DigitalOcean**	**Netlify, Vercel, Render**	**Gmail, Figma, Shopify**

"A cloud é o computador de outra pessoa, alugado à medida." Paga-se o que se usa.

FLAG

TEORIA

Cópias perto de quem visita



O que é

Uma rede de servidores (edge) com cópias dos ficheiros em centenas de cidades.

Porquê

Menos distância = menos tempo. E a origem não rebenta num pico de visitas.

Quem

Cloudflare, Fastly, Akamai. Netlify e Vercel já incluem uma.

FLAG

Onde publicar um site em 2026

Tipo	Para quê	Exemplos	Custo típico
Static hosting	Sites HTML/CSS/JS e SPAs (React): o vosso portfólio	GitHub Pages, Netlify, Vercel, Cloudflare Pages	Gratuito
Partilhado	WordPress e sites PHP simples, com email	PTisp, WebHS, Hostinger	Baixo
VPS	Controlo total de um servidor Linux: back-ends próprios	Hetzner, DigitalOcean, OVH	Médio
PaaS	Publicar aplicações sem gerir o servidor	Render, Railway, Fly.io	Gratuito → pago
Cloud	Aplicações de grande escala	AWS, Azure, Google Cloud	Por consumo

Neste curso: GitHub Pages no módulo Git (micro-site) e Vercel ou Netlify no projeto React final.

Fornecedores e preços indicativos: as ofertas mudam com frequência.

TEORIA

Do vosso computador ao mundo



Antes: FTP, ficheiro a ficheiro, para o servidor. Hoje: um git push e o site atualiza sozinho.

É o caminho que vão fazer: o micro-site HTML/CSS no GitHub Pages (módulo 3) e o projeto final em React na Vercel ou Netlify (módulo 8).

FLAG

EXERCÍCIO

Onde alojar?

1. O vosso portfólio em HTML e CSS
2. O site WordPress de um restaurante, com email
3. Uma API em Node.js com base de dados vossa
4. Uma app de bilhetes com picos de milhões de visitas
5. O projeto final em React deste curso

As opções

Static hosting
Partilhado
VPS
PaaS
Cloud

Pode haver mais do que uma resposta certa: justifiquem.

FLAG

SOLUÇÃO

Onde alojar: as respostas

1	O vosso portfólio em HTML e CSS	**Static hosting**	GitHub Pages: grátis, HTTPS e CDN.
2	O site WordPress de um restaurante, com email	**Partilhado**	Barato, com painel e email incluído.
3	Uma API em Node.js com base de dados vossa	**VPS ou PaaS**	VPS para controlo total; PaaS para não gerir o servidor.
4	Uma app de bilhetes com picos de milhões de visitas	**Cloud**	Escala à medida e paga-se o consumo.
5	O projeto final em React deste curso	**Static hosting**	Vercel ou Netlify, com deploy a cada push.

E o domínio? Regista-se à parte, num registrar, e liga-se ao alojamento pelo DNS (registos A ou CNAME).

FLAG

EXERCÍCIO 5.1 · 10 MIN

EXERCÍCIO

flag.pt + Enter: agora expliquem vocês

1 **DNS**

2 **Ligação**

3 **Pedido HTTP**

4 **Resposta**

5 **Mais pedidos**

6 **Render**

Cada formando explica dois passos ao microfone, por palavras suas. Os outros completam.

FLAG

SOLUÇÃO 5.1

SOLUÇÃO

flag.pt + Enter: o que acontece

01

DNS

O browser descobre o IP de flag.pt (cache ou resolver).

02

Ligação

Abre uma ligação TCP (ou QUIC) e negoceia a cifra TLS.

03

Pedido HTTP

GET / com cabeçalhos: idioma, cookies, formatos aceites.

04

Resposta

O servidor devolve 200 + HTML (talvez gerado por código e base de dados).

05

Mais pedidos

O HTML refere CSS, JS, imagens e fontes: novos pedidos, em paralelo.

06

Render

O browser constrói a página e desenha os pixels no ecrã.

Passos 1–4: arquitetura da Web (sessão 5)

5–6: front-end (hoje e o resto do curso)

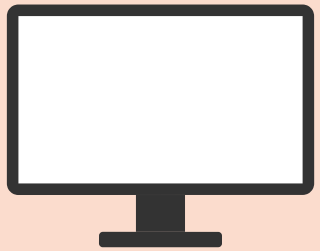
Tudo isto acontece, em regra, em menos de um segundo.

FLAG

HOJE

Abrir as duas pontas da conversa

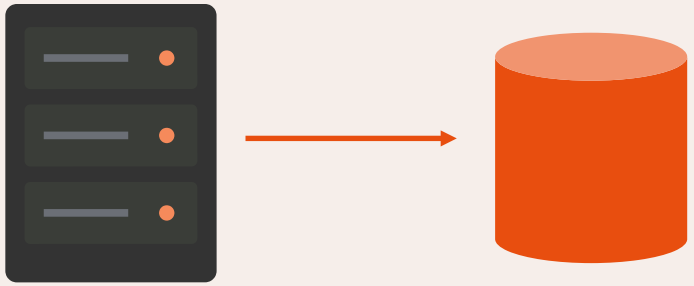
BROWSER · FRONT-END



HTML · CSS · JavaScript

`HTTP · JSON`

SERVIDOR · BACK-END



Node · Python · PHP · base de dados

Acabámos de ver o caminho. Agora abrimos as caixas das pontas.

Neste curso vivemos sobretudo na da esquerda, mas um bom front-end sabe o que acontece na da direita.

HOJE

O plano da sessão

1

Dois lados

Quem faz o quê, onde corre o código, estático vs dinâmico.

2

O front-end por dentro

HTML, CSS e JS por dentro, a árvore DOM, do código aos pixels.

3

Back-end e dados

APIs, JSON, bases de dados e o mapa completo.

4

Ferramentas e fecho

Terminal, setup para HTML e CSS e quiz do módulo.

Primeiro fechamos a sessão 5. Depois: teoria → exercício → solução (juntos)

Vamos ver o primeiro código "a sério" (HTML, CSS, JS e JSON), só para ler. Escrevê-lo é para os próximos módulos.

FLAG

PARTE 1

Dois lados do mesmo produto

responsabilidades · onde corre o código · confiança · estático vs dinâmico

TEORIA → EXERCÍCIOS 1.1 · 1.2, cada um seguido da solução

FLAG

Sala, empregado, cozinha e despensa

SALA

Front-end

O que o cliente vê e toca: layout, botões, formulários, animações.

`HTML · CSS · JS · React`

→

EMPREGADO

API

Leva o pedido à cozinha e traz a resposta. Só aceita o que está no menu.

`HTTP · JSON`

→

COZINHA

Back-end

Regras, autenticação, pagamentos, emails. O cliente não entra.

`Node · Python · PHP · Java · C#`

→

DESPENSA

Base de dados

Guarda os ingredientes de forma organizada e duradoura.

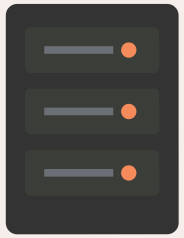
`PostgreSQL · MySQL · MongoDB`

Neste curso somos a sala

Construímos a experiência do utilizador e falamos com a cozinha através de APIs.

O front-end viaja; o back-end fica em casa

NO SERVIDOR (vosso)



Código do back-end
Passwords da base de dados
Chaves de pagamento
Dados de todos os clientes

Fica sempre aqui. O utilizador nunca o vê.

envia HTML, CSS e JS

NO BROWSER (do utilizador)



O código do front-end é descarregado e corre no computador ou telemóvel de quem visita.

Qualquer um o pode ler (Ctrl+U) e alterar (DevTools).

Regra: nunca pôr segredos no front-end

Passwords, chaves de APIs pagas, regras de preços: tudo o que não pode ser visto ou mudado fica no servidor.

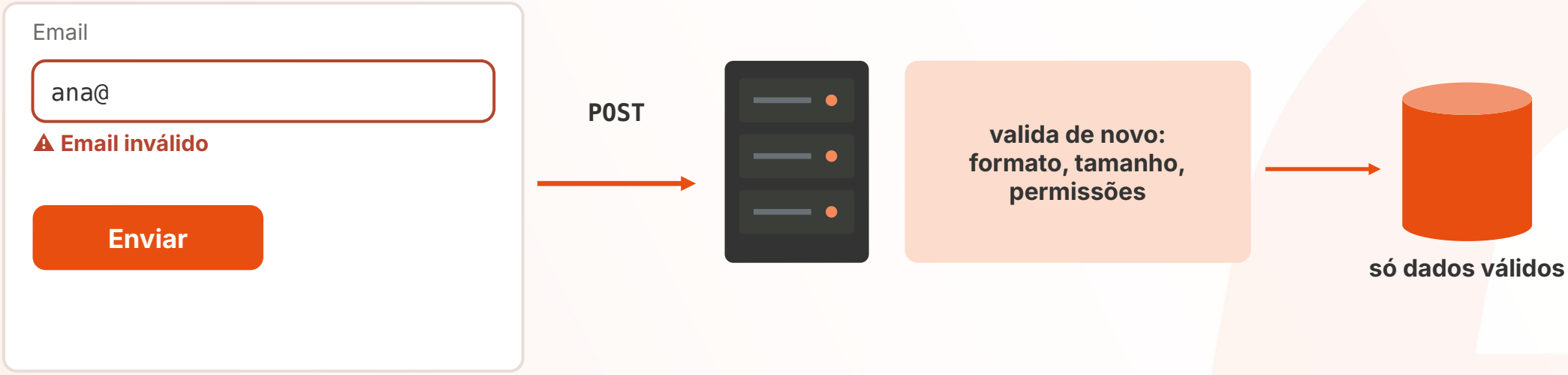
Quem faz o quê

	Front-end	Back-end
Onde corre	No browser do utilizador (o dispositivo dele)	No servidor (uma máquina que controlamos)
Preocupações	Layout, usabilidade, acessibilidade, responsividade, rapidez percebida	Segurança, regras de negócio, escala, integridade dos dados
Linguagens	HTML, CSS, JavaScript (e TypeScript)	JavaScript/TypeScript, Python, PHP, Java, C#, Go…
Ferramentas	VS Code, DevTools, npm, Vite, React/Angular/Vue	Express, Django, Laravel, Spring, .NET, bases de dados
Confiança	Nunca confiar: o utilizador pode alterar tudo	A última linha de defesa: valida tudo de novo

Full-stack = trabalhar nos dois lados

Com JavaScript/TypeScript dá para usar a mesma linguagem em ambos (Node.js no servidor).

Validar: conforto à frente, segurança atrás



Front-end: aviso imediato (UX, poupa um pedido)

Back-end: a decisão final (segurança)

O exemplo clássico: o preço no carrinho

No DevTools, qualquer um muda 49,90 € para 0,01 € antes de pagar. O servidor ignora o preço que vem do browser e recalcula-o a partir da base de dados.

TEORIA

Estático, dinâmico ou SPA

Estático

HTML pronto

→ browser

O servidor envia os ficheiros tal como estão. Igual para todos.

Ex.: portfólio, info.cern.ch, landing pages

Dinâmico

código + BD

→ HTML gerado

→ browser

A cada pedido, o servidor corre código, consulta a BD e gera o HTML.

Ex.: WordPress, lojas, fóruns

SPA

HTML vazio + JS

→ JS pede JSON

→ JS desenha

O servidor envia um HTML quase vazio e JS. O JS pede JSON à API e constrói a página no browser.

Ex.: Gmail, Figma, o vosso projeto React

Na prática misturam-se: frameworks como Next.js geram HTML no servidor e depois "acordam" a página com JS no browser.

FLAG

DEMO

Ver o que o servidor enviou

1 Ver o código-fonte

Ctrl+U (⌥⌘U no Mac) em info.cern.ch: é tudo o que o servidor enviou.

2 Inspecionar

Clique direito › Inspecionar: a página como está agora (depois do JS).

3 Desligar o JavaScript

DevTools › Ctrl+Shift+P (⌘⇧P) › "Disable JavaScript" › recarregar.

4 Comparar

Uma notícia continua legível? E o Google Maps? O que deixa de funcionar?

Um site estático ou bem feito continua a funcionar sem JS. Uma SPA mostra uma página vazia.

FLAG

EXERCÍCIO

Front-end, back-end ou os dois?

	Tarefa	Front	Back
1	Mudar a cor do botão "Comprar"		
2	Verificar se a password está certa		
3	Animação ao abrir o menu		
4	Enviar o email de confirmação da encomenda		
5	Guardar a encomenda		
6	Avisar "campo obrigatório" no formulário		
7	Calcular o total com os preços reais		
8	Modo escuro		
9	Redimensionar a foto de perfil enviada		
10	Filtrar uma lista que já está no ecrã		

A pergunta

Onde tem de acontecer? No browser de quem visita, no servidor, ou nos dois?

Pista

Pelo menos uma tem resposta "os dois".

SOLUÇÃO

Front-end, back-end ou os dois: as respostas

	Tarefa	Front	Back
1	Mudar a cor do botão "Comprar"	✓	
2	Verificar se a password está certa		✓
3	Animação ao abrir o menu	✓	
4	Enviar o email de confirmação da encomenda		✓
5	Guardar a encomenda		✓
6	Avisar "campo obrigatório" no formulário	✓	✓
7	Calcular o total com os preços reais		✓
8	Modo escuro	✓	
9	Redimensionar a foto de perfil enviada		✓
10	Filtrar uma lista que já está no ecrã	✓	

O 6 é dos dois

No front-end é conforto (aviso imediato); no back-end é segurança (valida sempre).

O 7

O browser pode mostrar uma estimativa, mas o preço a pagar vem da BD.

FLAG

EXERCÍCIO 1.2 · 5 MIN

EXERCÍCIO

Estático, dinâmico ou SPA?

1	Um portfólio em HTML e CSS no GitHub Pages	?	
2	Um blog em WordPress	?	
3	O Gmail	?	
4	info.cern.ch	?	
5	Uma loja online com stock e carrinho	?	
6	O projeto final em React deste curso	?	

Se tiverem dúvidas num, escolham e justifiquem: há sites que misturam.

FLAG

SOLUÇÃO

Estático, dinâmico ou SPA: as respostas

1. Um portfólio em HTML e CSS no GitHub Pages — **Estático**
2. Um blog em WordPress — **Dinâmico**
3. O Gmail — **SPA**
4. info.cern.ch — **Estático**
5. Uma loja online com stock e carrinho — **Dinâmico**
6. O projeto final em React deste curso — **SPA**

A loja (5) também pode ser uma SPA a falar com uma API. O que conta é perceber onde a página é construída.

FLAG

PARTE 2

O front-end por dentro

três camadas · HTML, CSS e JS por dentro · a árvore DOM · do código aos pixels

TEORIA → EXERCÍCIOS 2.1 · 2.2, cada um seguido da solução

FLAG

Estrutura, apresentação, comportamento

JavaScript

Comportamento

O que faz: reagir a cliques, validar, buscar dados, mudar a página.

≈ os músculos

CSS

Apresentação

Como se vê: cores, tipografia, espaçamento, layout, animação.

≈ a pele e a roupa

HTML

Estrutura e conteúdo

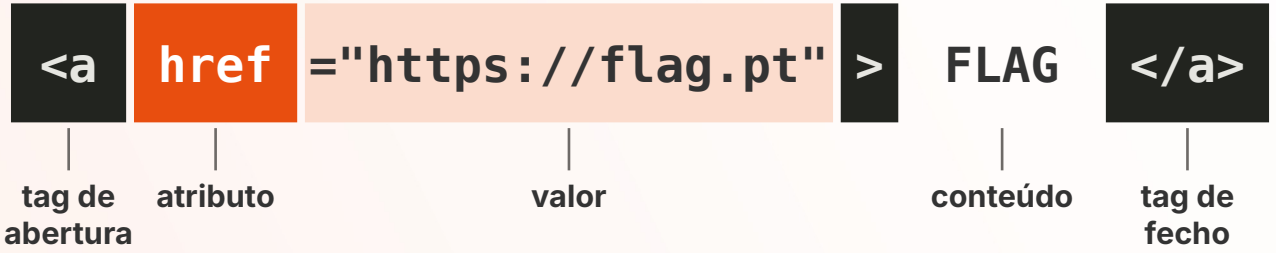
O que é cada coisa: título, parágrafo, imagem, botão, formulário.

≈ o esqueleto

▲ cada camada enriquece a de baixo

São as três linguagens que o browser percebe

Elementos: etiquetas à volta do conteúdo



tudo junto = um elemento (aqui, uma hiperligação)

Dentro uns dos outros

Os elementos encaixam como caixas: o body contém o h1 e o p.

O nome diz o significado

h1 = título principal, p = parágrafo, a = ligação, img = imagem.

A ESTRUTURA MÍNIMA

```
<!doctype html>
<html lang="pt">
  <head>
    <title>FLAG</title>
  </head>
  <body>
    <h1>Olá, Web!</h1>
    <p>O meu 1.º site.</p>
  </body>
</html>
```

head: informação sobre a página (título do separador).
body: o que se vê.

Regras: a quem, o quê, quanto



propriedade: valor; = uma declaração

Uma regra pode ter muitas declarações, uma por linha.

TRÊS FORMAS DE LIGAR CSS AO HTML

No elemento

```
<h1 style="color: red">
```

evitar

Na página

```
<style> h1 { … } </style>
```

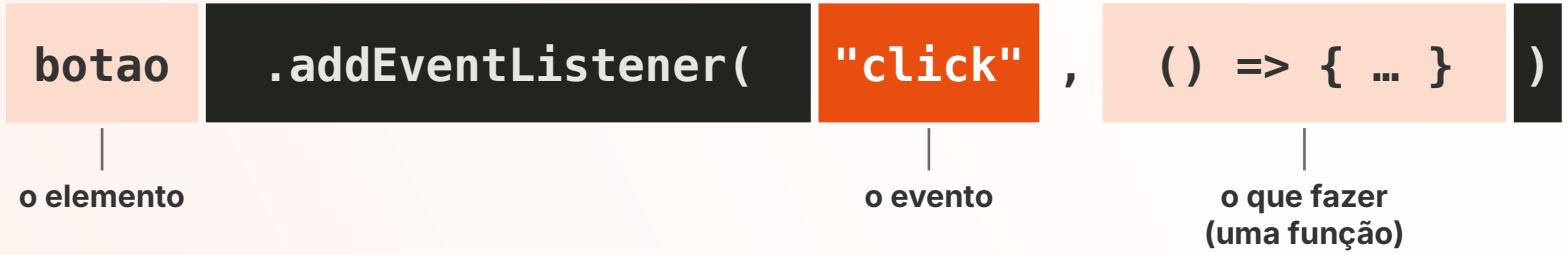
para testes

Num ficheiro

```
<link rel="stylesheet"
      href="estilos.css">
```

o recomendado

Quando acontecer isto, faz aquilo



JÁ CONHECEM ISTO DO PORTUGOL

Portugol	JavaScript
`inteiro idade <- 18`	`let idade = 18;`
`escrever "Olá"`	`console.log("Olá");`
`se idade >= 18 entao … fimse`	`if (idade >= 18) { … }`
`para i de 1 ate 10 passo 1`	`for (let i = 1; i <= 10; i++)`

A lógica é a mesma

Variáveis, decisões e ciclos. Muda a sintaxe e aparece uma coisa nova: a página (o DOM) e os eventos do utilizador.

O mesmo botão em três linguagens

HTML · O QUE É

```
<button class="cta">
  Inscrever
</button>
```

CSS · COMO SE VÊ

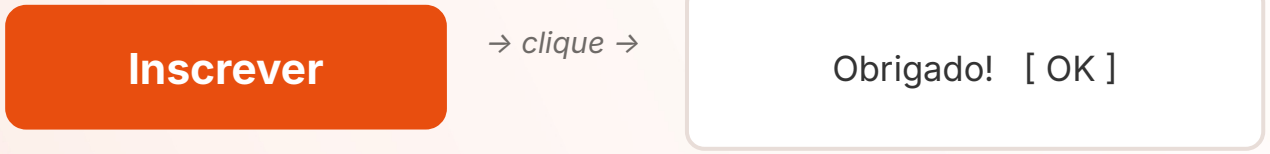
```
.cta {
  background: #E84E0F;
  color: white;
  padding: 1rem 2rem;
  border-radius: 8px;
}
```

JS · O QUE FAZ

```
const cta =
  document.querySelector(".cta");

cta.addEventListener("click",
  () => alert("Obrigado!")
);
```

RESULTADO



Separação de responsabilidades

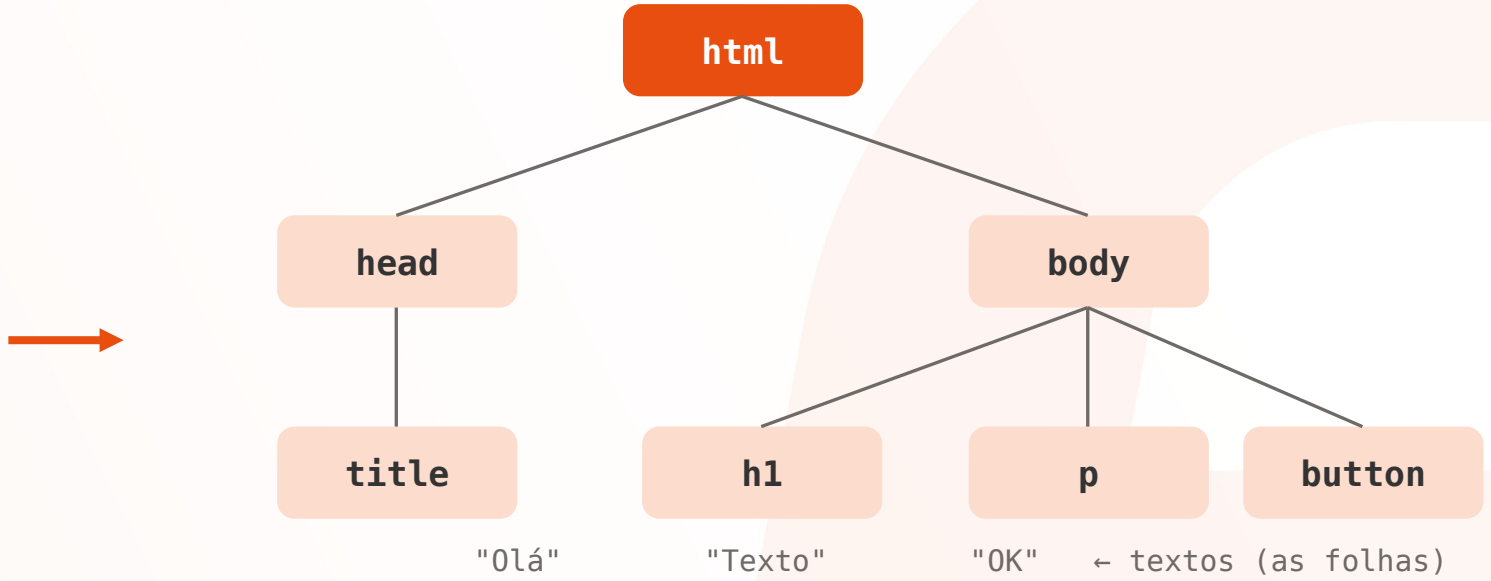
Cada camada muda sem partir as outras. A class "cta" é a ponte.

TEORIA

O browser transforma o HTML numa árvore

HTML

```
<html>
  <head>
    <title>FLAG</title>
  </head>
  <body>
    <h1>Olá</h1>
    <p>Texto</p>
    <button>OK</button>
  </body>
</html>
```



Cada caixa é um nó, com pai e filhos. O JavaScript lê e muda a árvore (módulo 4); o React gere-a por nós (módulos 5–6). O DevTools › Elements mostra-a ao vivo.

FLAG

TEORIA

Como o browser desenha uma página

01

HTML → DOM

Lê o HTML e constrói a árvore de elementos.

02

CSS → CSSOM

Lê o CSS e calcula os estilos de cada elemento.

03

Render tree

Junta os dois: só o que é visível, já com estilos.

04

Layout

Calcula a posição e o tamanho de cada caixa.

05

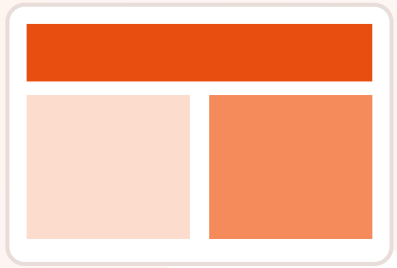
Paint

Pinta os pixels no ecrã.

E o JavaScript?

Pode mudar o DOM e os estilos a qualquer momento; o browser repete os passos necessários. Muitas mudanças seguidas = página lenta.

layout: caixas



paint: pixels

FLAG

DEMO

Mexer numa página que não é nossa

1. **Elements**
   Inspecionar o título da página e mudar o texto (duplo clique).

2. **Styles**
   No mesmo elemento, mudar a cor ou o tamanho da letra.

3. **Console**
   Mudar o título com JavaScript (código ao lado).

4. **Recarregar**
   Tudo volta ao original. Porquê?

CONSOLA · SESSAO-06-SNIPPETS.TXT

```
// escolher o 1.º h1 da página
const titulo =
  document.querySelector("h1");

titulo.textContent = "Olá, FLAG!";
titulo.style.color = "green";
```

Resposta ao 4

As alterações são só no vosso browser, na vossa cópia do DOM. O servidor não sabe de nada.

FLAG

EXERCÍCIO 2.1 · 5 MIN

EXERCÍCIO

HTML, CSS ou JavaScript?

1. Acrescentar um parágrafo novo
2. Pôr os títulos a verde
3. Mostrar uma mensagem ao clicar num botão
4. Pôr três cartões lado a lado
5. Adicionar uma imagem com texto alternativo
6. Avisar se o email está mal escrito enquanto se escreve
7. Mudar o tipo de letra do site todo
8. Carregar a lista de cursos de uma API
9. Criar uma ligação para a página de contactos

No chat: "1-HTML, 2-...". A pergunta a fazer: é conteúdo, aparência ou comportamento?

FLAG

SOLUÇÃO 2.1

SOLUÇÃO

HTML, CSS ou JavaScript: as respostas

1. Acrescentar um parágrafo novo — **HTML**
2. Pôr os títulos a verde — **CSS**
3. Mostrar uma mensagem ao clicar num botão — **JS**
4. Pôr três cartões lado a lado — **CSS**
5. Adicionar uma imagem com texto alternativo — **HTML**
6. Avisar se o email está mal escrito enquanto se escreve — **JS**
7. Mudar o tipo de letra do site todo — **CSS**
8. Carregar a lista de cursos de uma API — **JS**
9. Criar uma ligação para a página de contactos — **HTML**

O 6 tem truque: o HTML já valida emails ao enviar (type="email"); avisar enquanto se escreve é JS.

EXERCÍCIO 2.2 · 8 MIN

EXERCÍCIO

Desenhar a árvore DOM

PAGINA.HTML

```
<body>
  <header>
    <h1>FLAG</h1>
    <nav>
      <a>Cursos</a>
      <a>Contactos</a>
    </nav>
  </header>
  <main>
    <p>Olá!</p>
  </main>
</body>
```

O que fazer

Em papel ou no quadro do Teams: desenhem a árvore, do body até às folhas. Quem é pai de quem? Quantos filhos tem o nav?

Regra

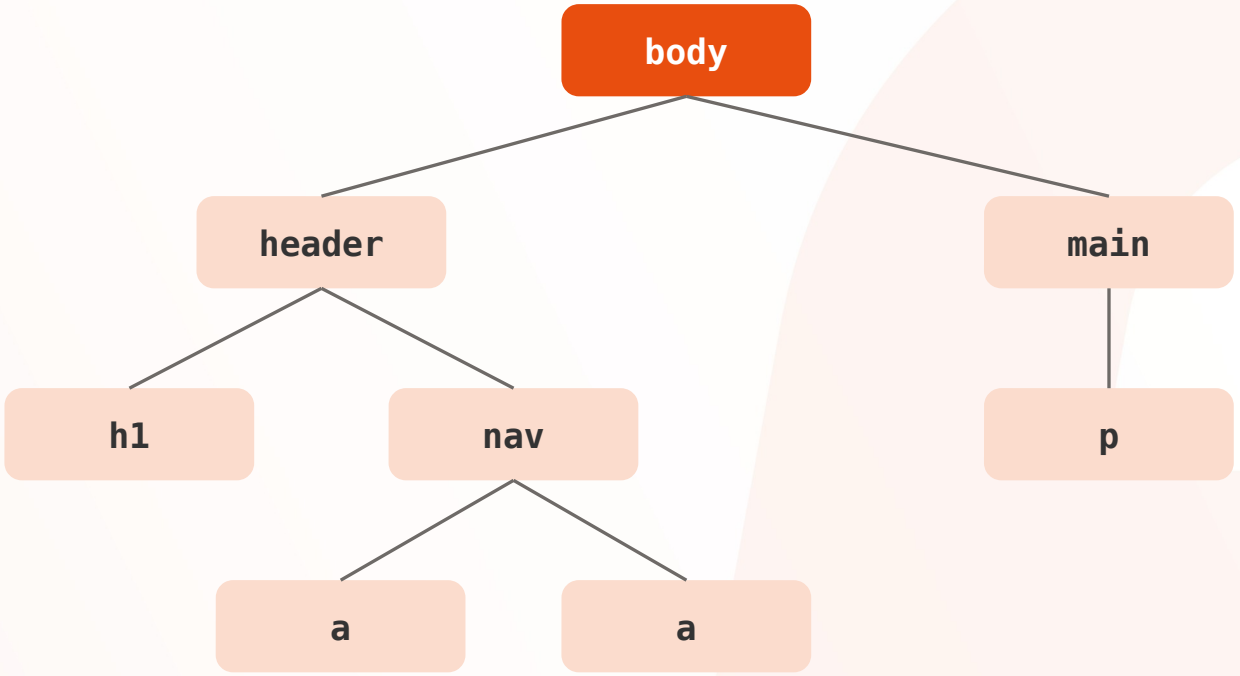
Um elemento é filho do elemento que o envolve diretamente. A indentação do código ajuda (cada nível, 2 espaços).

SOLUÇÃO

A árvore da página

PAGINA.HTML

```
<body>
  <header>
    <h1>FLAG</h1>
    <nav>
      <a>Cursos</a>
      <a>Contactos</a>
    </nav>
  </header>
  <main>
    <p>Olá!</p>
  </main>
</body>
```



O nav tem 2 filhos (os dois a). O h1 e o nav são irmãos: têm o mesmo pai, o header.

FLAG

TEORIA

Para além das três camadas

Frameworks

React (módulos 5–6), Angular, Vue, Svelte: componentes reutilizáveis e estado organizado.

Tooling

npm para pacotes, Vite para desenvolver e gerar a versão final (build).

TypeScript

JavaScript com tipos. Padrão na indústria: vamos espreitá-lo no fim do curso.

Design e UX

Ler designs em Figma, design systems, acessibilidade (WCAG) como requisito.

Responsivo

Do telemóvel ao ecrã grande com o mesmo código: media e container queries.

Performance

Core Web Vitals do Google: rapidez, estabilidade e resposta da página.

FLAG

PARTE 3

Back-end, APIs e dados

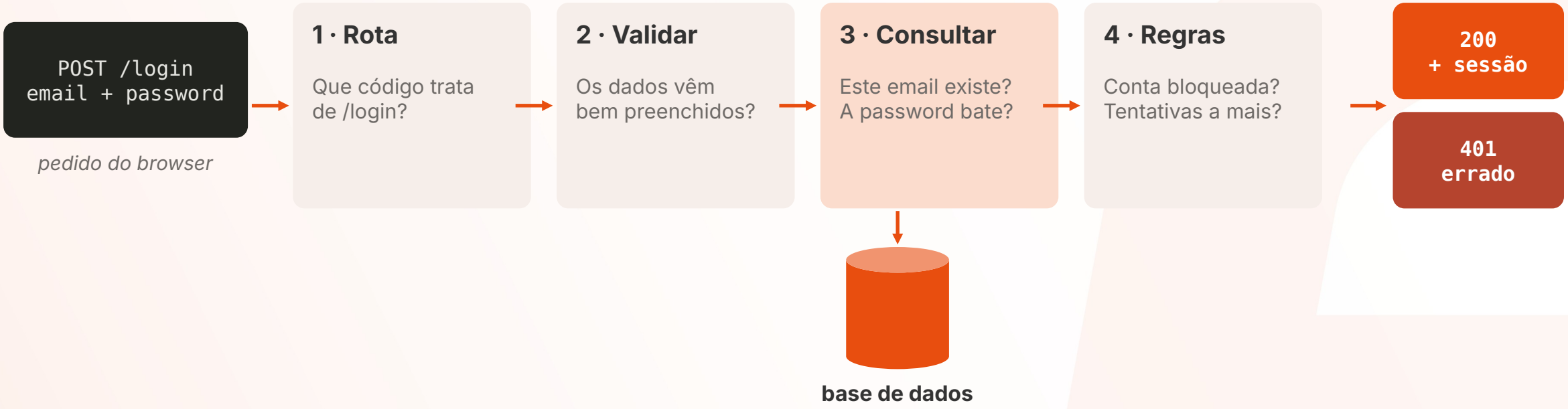
o que faz o servidor · APIs e JSON · bases de dados · o mapa completo

TEORIA → EXERCÍCIOS 3.1 · 3.2 (e 3.3 para os mais expeditos), cada um seguido da solução

FLAG

TEORIA

O caminho de um login



Tudo isto o utilizador não vê: só recebe a resposta

O back-end também envia emails, processa pagamentos, gera PDFs, fala com outras APIs e guarda registos (logs).

FLAG

Linguagens e frameworks mais comuns

Linguagem	Frameworks	Onde é forte
JavaScript / TypeScript	Node.js + Express, NestJS, Next.js	Mesma linguagem do front-end; APIs e tempo real
Python	Django, FastAPI, Flask	Dados, IA e machine learning, automação
PHP	Laravel, Symfony, WordPress	Grande parte da Web existente; alojamento barato
Java	Spring Boot	Empresas, banca, sistemas de grande escala
C#	ASP.NET Core	Ecossistema Microsoft, empresas
Go	biblioteca padrão, Gin	Serviços rápidos, infraestrutura cloud

Não há "a melhor"

Escolhe-se pela equipa, pelo problema e pelo ecossistema. Todas falam HTTP e JSON com o front-end: para nós, é igual.

TEORIA

O menu que o back-end oferece

API da escola · menu

GET	`/cursos`	*lista de cursos*
GET	`/cursos/42`	*um curso*
POST	`/inscricoes`	*inscrever-me*
PATCH	`/formandos/7`	*mudar o meu email*
DELETE	`/inscricoes/3`	*anular inscrição*

API

Application Programming Interface: o contrato do que se pode pedir, como pedir e o que vem de volta.

Endpoint

Cada linha do menu: um método HTTP + um caminho.

REST

O estilo mais comum: URLs para as coisas (recursos), métodos para as ações. Os métodos da sessão 5!

FLAG

TEORIA

O formato em que os dados viajam

```
{
  "nome": "Ana",
  "idade": 29,
  "ativa": true,
  "modulos": ["HTML", "Git"],
  "morada": { "cidade": "Lisboa" },
  "telefone": null
}
```

- **texto (entre aspas)**
- **número**
- **booleano (true/false)**
- **lista (array) [ ]**
- **objeto dentro de objeto**
- **sem valor**

Pares chave: valor

Chaves sempre entre aspas. Separados por vírgulas.

Parece um objeto JavaScript porque nasceu dele (JavaScript Object Notation).

Tipos como no Portugol: texto, inteiro/real, lógico, vetor.

FLAG

DEMO

Pedir dados a uma API, na consola

CONSOLA DO BROWSER · SESSAO-06-SNIPPETS.TXT

```
const url =
  "https://jsonplaceholder.typicode.com/users/1";

const res = await fetch(url);   // pedido GET
const user = await res.json();  // texto → objeto

console.log(user.name);
// "Leanne Graham"
```

A RESPOSTA (JSON, RESUMIDA)

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz",
  "address": {
    "city": "Gwenborough"
  }
}
```

Depois, no separador Network › Fetch/XHR

Vemos o mesmo pedido: GET, 200, Content-Type: application/json. Uma API pública e gratuita, feita para treinar.

FLAG

TEORIA

Tabelas ou documentos

RELACIONAL (SQL) · POSTGRESQL, MYSQL, SQLITE

id	nome	modulo_id
1	Ana	2
2	Bruno	4
3	Carla	2

formandos

id	modulo
2	HTML e CSS
4	JavaScript

modulos

```
SELECT nome FROM formandos
WHERE modulo_id = 2;   -- Ana, Carla
```

DOCUMENTAL (NOSQL) · MONGODB

```
// coleção formandos
{
  "_id": 1,
  "nome": "Ana",
  "modulos": ["HTML e CSS", "Git"],
  "portfolio": { "github": "ana-dev" }
}
```

Regra de ouro: o browser nunca fala diretamente com a base de dados

Passa sempre pelo back-end. Supabase e Firebase parecem exceção, mas expõem uma API com regras de acesso.

FLAG

Quatro ações, três linguagens

Ação (CRUD)	No browser	HTTP	SQL	Exemplo
Create · criar	enviar um formulário	`POST`	`INSERT`	inscrever-me num curso
Read · ler	abrir uma página	`GET`	`SELECT`	ver os meus cursos
Update · alterar	guardar o perfil	`PUT / PATCH`	`UPDATE`	mudar o email
Delete · apagar	clicar em "remover"	`DELETE`	`DELETE`	anular a inscrição

Quase todas as aplicações são CRUD com bom aspeto

Lojas, redes sociais, o Moodle, a app do banco: criar, ler, alterar e apagar coisas. Vão reconhecer este padrão em todo o lado.

TEORIA

Uma aplicação full-stack e onde entra cada módulo

BROWSER · FRONT-END

- **HTML e CSS** — módulo 2
- **JavaScript** — módulo 4
- **React** — módulos 5 e 6

`HTTP · JSON` →

SERVIDOR · API

Node.js, Python, PHP...

regras de negócio
autenticação
validação

→



SQL ou NoSQL

Git e GitHub (módulo 3): versionar e publicar tudo isto · Projeto (módulo 8): juntar tudo numa aplicação real, publicada

FLAG

EXERCÍCIO

Ler um JSON

CURSO.JSON

```
{
  "nome": "Front-End Web Development",
  "horas": 183,
  "ativo": true,
  "formador": {
    "nome": "João",
    "cidade": "Lisboa"
  },
  "modulos": ["HTML e CSS", "Git",
              "JavaScript", "React"],
  "vagas": null
}
```

1. Quantos módulos tem o curso?
2. Qual o nome do formador? E o "caminho" até lá?
3. De que tipo é "ativo"?
4. Qual o 1.º módulo e em que índice está?
5. "horas" é texto ou número? Como se sabe?

FLAG

SOLUÇÃO

Ler um JSON: as respostas

CURSO.JSON

```
{
  "nome": "Front-End Web Development",
  "horas": 183,
  "ativo": true,
  "formador": {
    "nome": "João",
    "cidade": "Lisboa"
  },
  "modulos": ["HTML e CSS", "Git",
              "JavaScript", "React"],
  "vagas": null
}
```

1. Quantos módulos tem o curso?
   4: é uma lista com 4 elementos.
2. Qual o nome do formador? E o "caminho" até lá?
   "João" · curso.formador.nome
3. De que tipo é "ativo"?
   Booleano (true / false).
4. Qual o 1.º módulo e em que índice está?
   "HTML e CSS", índice 0 (como nos vetores).
5. "horas" é texto ou número? Como se sabe?
   Número: não tem aspas.

FLAG

EXERCÍCIO 3.2 · 8 MIN

EXERCÍCIO

Desenhar a API de uma biblioteca

1	Listar todos os livros	`MÉTODO /caminho`
2	Ver o livro 12	`MÉTODO /caminho`
3	Adicionar um livro novo	`MÉTODO /caminho`
4	Corrigir o ano do livro 12	`MÉTODO /caminho`
5	Apagar o livro 12	`MÉTODO /caminho`
6	Livros de um autor	`MÉTODO /caminho`

Pistas

O recurso é "livros", no plural.

Um livro em concreto: o id no caminho.

O 6 lembra a query string da sessão 5.

É a API da app "biblioteca" que vão fazer no módulo 4.

SOLUÇÃO

A API da biblioteca

1	Listar todos os livros	GET	`/livros`
2	Ver o livro 12	GET	`/livros/12`
3	Adicionar um livro novo	POST	`/livros`
4	Corrigir o ano do livro 12	PATCH	`/livros/12`
5	Apagar o livro 12	DELETE	`/livros/12`
6	Livros de um autor	GET	`/livros?autor=saramago`

Um caminho, vários métodos

/livros/12 serve para ver, alterar e apagar: o método é que diz a ação.

PUT também serve no 4, se enviarmos o livro todo.

FLAG

EXERCÍCIO

O vosso primeiro fetch

PONTO DE PARTIDA (DEMO)

```
const url =
  "https://jsonplaceholder.typicode.com/users/1";

const res = await fetch(url);
const user = await res.json();
console.log(user.name);
```

A Mudem para /todos/1 e mostrem o título (title) e se está concluída (completed).

B Peçam /users (sem número). Quantos utilizadores vêm na lista?

C Do utilizador 1, mostrem só a cidade (está dentro de address).

Na consola do browser, em jsonplaceholder.typicode.com

Pista para o B: uma lista tem .length (como o tamanho de um vetor).

FLAG

SOLUÇÃO

O vosso primeiro fetch: as respostas

A

```
const res = await fetch(
  ".../todos/1");
const t = await res.json();
console.log(t.title,
            t.completed);
// "delectus aut autem" false
```

B

```
const res = await fetch(
  ".../users");
const users = await res.json();
console.log(users.length);
// 10
```

C

```
const res = await fetch(
  ".../users/1");
const u = await res.json();
console.log(u.address.city);
// "Gwenborough"
```

O ponto liga as chaves: u.address.city

É o "caminho" do exercício 3.1. Os textos estão em latim de propósito: são dados falsos para treinar.

FLAG

PARTE 4

Ferramentas e fecho do módulo

saídas profissionais · a bancada de trabalho · terminal · setup · quiz

TEORIA → SETUP JUNTOS → QUIZ DO MÓDULO

FLAG

Para onde pode levar esta especialização

1

Front-end Developer

Interfaces web e SPAs com HTML, CSS, JS e frameworks. O foco deste curso.

2

Web Developer

Sites e aplicações completas, muitas vezes com um CMS e algum back-end.

3

Full-stack Developer

Front + back. Próximo passo natural: Node.js e bases de dados.

4

UI Engineer · Design Systems

A ponte entre design e código: componentes, acessibilidade, consistência.

O portfólio é o vosso CV

O micro-site, as apps JS e o projeto React é o que vão mostrar em entrevistas. Tratem o GitHub desde já como uma montra.

As ferramentas de todos os dias

1

Browser + DevTools

Ver, inspecionar e depurar. Chrome ou Firefox.

todos os módulos

2

VS Code

O editor. Extensões: Live Server, Prettier.

todos os módulos

3

Terminal

Dar ordens ao computador por texto.

módulos a partir do 3

4

Node.js + npm

Correr JS fora do browser e instalar pacotes.

módulos 4 a 6

5

Git + GitHub

Histórico de versões e publicar.

módulos 3 em diante

Tudo grátis e igual em Windows e macOS

Instalamos já hoje: no módulo HTML e CSS, na quarta-feira, começamos logo a escrever código.

TEORIA

O mínimo para sobreviver

Para…	macOS (Terminal)	Windows (PowerShell)
saber onde estou	`pwd`	`pwd`
ver o que está na pasta	`ls`	`ls  (ou dir)`
entrar numa pasta	`cd flag-frontend`	`cd flag-frontend`
voltar uma pasta acima	`cd ..`	`cd ..`
criar uma pasta	`mkdir 02-html-css`	`mkdir 02-html-css`
abrir a pasta no VS Code	`code .`	`code .`
limpar o ecrã	`clear`	`clear  (ou cls)`

Sem acentos nem espaços nos nomes de pastas: poupam-se muitos problemas no terminal.

Tab completa

Escrevam as primeiras letras de uma pasta e carreguem em Tab.

code . no Mac

No VS Code: ⌘⇧P › "Shell Command: Install 'code' command in PATH".

FLAG

ANTES DE QUARTA-FEIRA

TEORIA

Preparar o computador para HTML e CSS

1

Browser moderno

Chrome ou Firefox atualizado. Vamos viver no DevTools.

2

VS Code

code.visualstudio.com + extensões: Live Server (ou Live Preview), Prettier.

3

Node.js LTS

nodejs.org, versão LTS. Confirmar: node -v e npm -v.

4

Git

git-scm.com (Windows) ou xcode-select --install (macOS). Confirmar: git --version.

5

Conta GitHub

github.com, com um nome profissional: vai ser o vosso portfólio.

6

Pasta de trabalho

flag-frontend/ com uma subpasta por módulo. Sem espaços nem acentos.

DEMO

Confirmar que está tudo instalado

TERMINAL · SESSAO-06-SNIPPETS.TXT

```
# versões (cada um cola o resultado no chat)
node -v           # v24.x ou mais recente
npm -v
git --version
code --version

# a pasta do curso
mkdir flag-frontend
cd flag-frontend
mkdir 02-html-css
code .
```

A ESTRUTURA QUE QUEREMOS

```
flag-frontend/
├── 01-intro-programacao/
├── 02-html-css/
├── 03-git-github/
├── 04-javascript/
└── …
```

Quem encalhar, partilha o ecrã

Os bloqueios típicos: permissões no Windows, o "code" que não é reconhecido, o terminal aberto antes de instalar (fechar e abrir de novo).

FLAG

QUIZ DO MÓDULO · 8 MIN

EXERCÍCIO

Seis perguntas para fechar o módulo

Q1 Que camada trata das cores e do layout?

- A) HTML
- B) JavaScript
- C) CSS
- D) JSON

Q2 Onde se faz a validação que garante segurança?

- A) Só no front-end
- B) No back-end, sempre
- C) Só na BD
- D) Não é preciso

Q3 Em que formato uma API costuma devolver dados?

- A) HTML
- B) CSS
- C) PDF
- D) JSON

Q4 O que faz o DNS?

- A) Cifra a ligação
- B) Traduz nomes em IPs
- C) Guarda páginas
- D) Desenha a página

Q5 Que método HTTP usamos para criar?

- A) GET
- B) POST
- C) DELETE
- D) HEAD

Q6 Num algoritmo, o que é um vetor?

- A) Um ciclo
- B) Uma decisão
- C) Uma variável com vários valores
- D) Um erro

No chat: "1-C, 2-...". Q6 é da primeira metade do módulo!

FLAG

SOLUÇÃO

As respostas

Q1 Que camada trata das cores e do layout?

- A) HTML
- B) JavaScript
- **C) CSS**
- D) JSON

Q2 Onde se faz a validação que garante segurança?

- A) Só no front-end
- **B) No back-end, sempre**
- C) Só na BD
- D) Não é preciso

Q3 Em que formato uma API costuma devolver dados?

- A) HTML
- B) CSS
- C) PDF
- **D) JSON**

Q4 O que faz o DNS?

- A) Cifra a ligação
- **B) Traduz nomes em IPs**
- C) Guarda páginas
- D) Desenha a página

Q5 Que método HTTP usamos para criar?

- A) GET
- **B) POST**
- C) DELETE
- D) HEAD

Q6 Num algoritmo, o que é um vetor?

- A) Um ciclo
- B) Uma decisão
- **C) Uma variável com vários valores**
- D) Um erro

RESUMO DO MÓDULO

Introdução à Programação: concluído

S1–S2

Pensar como programador

Pensamento computacional, algoritmos, os primeiros programas em Portugol.

S3–S4

Escrever algoritmos

Variáveis, operadores, decisões, ciclos, vetores e teste de mesa.

S5–S6

Perceber a Web

Internet, DNS, HTTP, alojamento, front-end vs back-end, APIs e dados.

A lógica dos algoritmos volta no módulo 4, em JavaScript.

O que vimos da Web hoje e na sexta é o mapa para os módulos 2 a 8.

FLAG

ATÉ QUARTA-FEIRA

Preparar o módulo HTML e CSS

1

Setup completo

Tudo o que ficou por instalar hoje. node -v, git --version e code . a funcionar.

2

Conta GitHub

Com nome profissional. Enviem o link no fórum do Moodle.

3

Opcional

MDN · "How the web works" e o vídeo Warriors of the Net (Moodle): o resumo destas duas sessões.

Próximo módulo · HTML e CSS · 4.ª, 14 out, 9:30

Começamos logo a escrever a primeira página. Tragam o computador com o setup feito.

FLAG

FLAG

www.flag.pt

LISBOA Edifício Mirage, R. Dr. Eduardo Neves, Nº3, 1050-077 Lisboa email: querosabermais@flag.pt Tel. 213 560 606

PORTO Rua Oliveira Monteiro, Nº 168, 4050–438 Porto email: querosabermais@porto.flag.pt Tel. 226 094 789

uma marca

Rumos training | Knowledge sharing